

✓ Hydrogen Atom

- **Author:** Boran Aybak Kilic
- **Date:** 05.02.2026
- **Time spent on this assignment:** 6 hours

```

1 import numpy as np
2 from numpy import *
3 import matplotlib.pyplot as plt
4 !pip install --upgrade scipy
5 import scipy
6 import scipy.sparse
7 import scipy.sparse.linalg
8 import plotly.graph_objects as go
9 from scipy.constants import physical_constants
10 import scipy.special as sp
11 from mpl_toolkits.mplot3d import Axes3D
12 import math

```

```

Requirement already satisfied: scipy in /usr/local/lib/python3.12/di
Collecting scipy
  Downloading scipy-1.17.1-cp312-cp312-manylinux_2_27_x86_64.manylin
    62.1/62.1 kB 1.8 MB/s
Requirement already satisfied: numpy<2.7,>=1.26.4 in /usr/local/lib/
Downloading scipy-1.17.1-cp312-cp312-manylinux_2_27_x86_64.manylinux
    35.2/35.2 MB 13.0 MB/s e
Installing collected packages: scipy
  Attempting uninstall: scipy
    Found existing installation: scipy 1.16.3
    Uninstalling scipy-1.16.3:
      Successfully uninstalled scipy-1.16.3
  Successfully installed scipy-1.17.1

```

Throughout this exercise, we are going to work in atomic units where

$\hbar = m = e = 1/(4\pi\epsilon_0) = 1$. This gives us the Bohr radius which is also equal to 1 in these units (and 0.529×10^{-10} m in SI units). All distances in this unit will be measured in Bohr radii.

✓ Exercise 1. Plotting the Hydrogen Atom Orbitals

In this exercise, we are going to plot the orbitals of the Hydrogen atom starting from the relevant solutions that you found analytically. In future exercises, we will see how to get these results from scratch without using the analytic solutions.

Recall that the Hydrogen atom wave-function is

$$\Psi_{nlm}(r, \theta, \phi) = R_{nl}(r)Y_{lm}(\theta, \phi)$$

✓ a. Radial Function

Let us start by getting the radial wave-function of the Hydrogen atom working. Recall that the radial wave-function is

$$R_{nl}(r) = \frac{u_{nl}(r)}{r} = N \exp(-\rho/2) \rho^l L_{2l+1}^{n-l-1}(\rho)$$

where

$$\rho = \frac{2r}{na_0}$$

and

$$N = \sqrt{\left(\frac{2}{na_0}\right)^3 \frac{(n-l-1)!}{2n(n+l)!}}$$

and L is a special function which can be generated by doing

`L=sp.genlaguerre(n-l-1, 2*l+1)`. Note that this just returns the python function. Later you will have to call `L(rho)`.

Write a function `radial_function(n, l, r)` which takes n , l , and r and returns $R_{nl}(r)$. Recall we are in units where $a_0 = 1$.

Let's start by evaluating this on a one-dimensional grid of positions

`r=np.linspace(0.001, 45, 1000)`.

Plot on the same plot, $u_{nl}(r)$ (notice this is different from R_{nl} by a factor of r) for

- $(n, \ell) = (1, 0)$
- $(n, \ell) = (2, 0)$
- $(n, \ell) = (3, 0)$

How do the number of nodes (times when it hits zero) correspond to the value of n ?

Now on a separate plot make

- $(n, \ell) = (3, 2)$
- $(n, \ell) = (3, 1)$
- $(n, \ell) = (3, 0)$

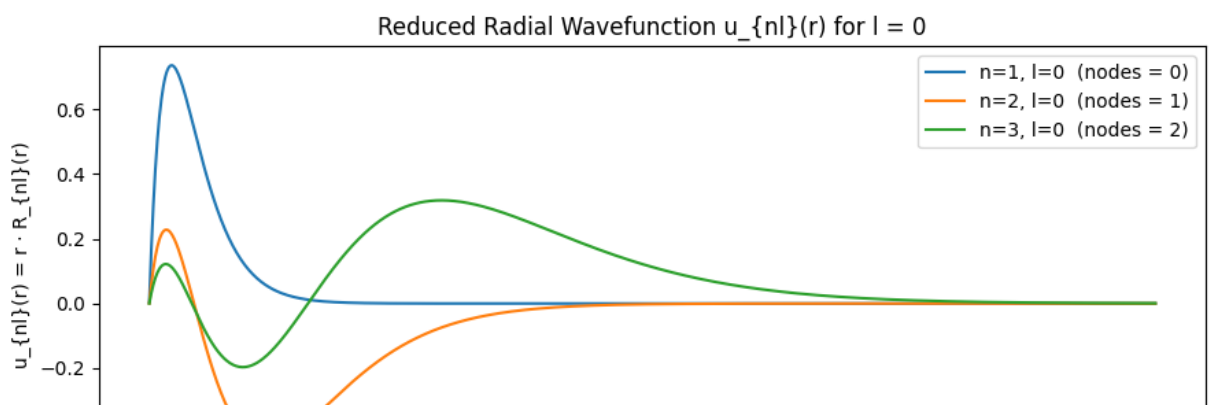
Notice how the larger values of l are pushed further away from the origin. This is coming from the additional force due to the fake potential.

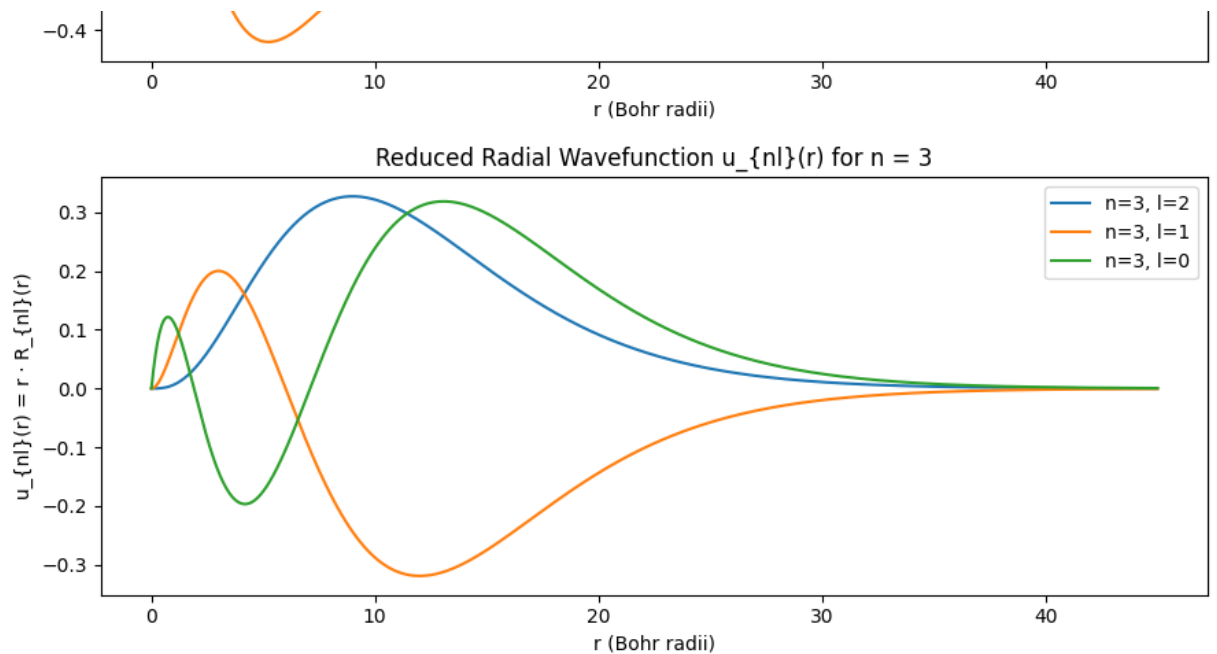
? Answer (start)

```

1 def radial_function(n, l, r):
2     """Hydrogen radial wavefunction  $R_{nl}(r)$  in atomic units (a.u.)
3     rho = 2.0 * r / n
4     L_poly = sp.genlaguerre(n - l - 1, 2 * l + 1)
5     norm = np.sqrt((2.0 / n)**3 * math.factorial(n - l - 1) /
6                    (2 * n * math.factorial(n + l)))
7     return norm * np.exp(-rho / 2.0) * rho**l * L_poly(rho)
8
9 r = np.linspace(0.001, 45, 1000)
10
11 # Plot  $u_{nl}(r) = r * R_{nl}(r)$  for  $l=0$  and  $n=1,2,3$ 
12 plt.figure(figsize=(9, 4))
13 for n, l in [(1, 0), (2, 0), (3, 0)]:
14     u = r * radial_function(n, l, r)
15     plt.plot(r, u, label=f'n={n}, l={l} (nodes = {n - l - 1})')
16 plt.xlabel('r (Bohr radii)')
17 plt.ylabel('u_{nl}(r) = r · R_{nl}(r)')
18 plt.title('Reduced Radial Wavefunction u_{nl}(r) for l = 0')
19 plt.legend()
20 plt.tight_layout()
21 plt.show()
22
23 # Plot u_{nl}(r) for n=3 and varying l
24 plt.figure(figsize=(9, 4))
25 for n, l in [(3, 2), (3, 1), (3, 0)]:
26     u = r * radial_function(n, l, r)
27     plt.plot(r, u, label=f'n={n}, l={l}')
28 plt.xlabel('r (Bohr radii)')
29 plt.ylabel('u_{nl}(r) = r · R_{nl}(r)')
30 plt.title('Reduced Radial Wavefunction u_{nl}(r) for n = 3')
31 plt.legend()
32 plt.tight_layout()
33 plt.show()
34

```





The number of nodes in $u_{nl}(r)$ equals $n-l-1$ (so $n=1$ has 0 nodes and $n=3$ has 2 nodes for $l=0$); higher l pushes the wavefunction away from the origin due to the centrifugal barrier in the effective potential.**

✓ Answer (end)

It should also be that the different $u_{nl}(r)$ at fixed l are orthogonal to each other. Check this by verifying that `u31.conjugate() @ u21.T=0` where you get `u` by multiplying your radial function by r .

? Answer (start)

```

1 dr = r[1] - r[0]
2 u21 = r * radial_function(2, 1, r)
3 u31 = r * radial_function(3, 1, r)
4 overlap = u31.conjugate() @ u21 * dr
5 print(f"<u31 | u21> = {overlap:.6e}    (should be ≈ 0, confirming

```

```

<u31 | u21> = 5.057857e-11    (should be ≈ 0, confirming orthogonality)

```

✓ Answer (end)

✓ b. Hydrogen Orbitals

We can now combine the radial wave-function with the spherical Harmonics to write down the Hydrogen orbitals

$$\Psi_{nlm}(x, y, z) = R_{nl}(x, y, z)Y_{lm}(\theta, \phi)$$

We are going to want to evaluate these orbitals on the entire space. Therefore, we will start by putting together a three-dimensional grid of the (x,y,z) positions.

```

def HydrogenGrid(g=10,numGridPoints=50):
    p=np.linspace(-g,g,numGridPoints)
    h=p[1]-p[0]
    x,y,z=np.meshgrid(p,p,p)
    return x,y,z,h

```

which you can then get the grid from by doing `x,y,z,h=HydrogenGrid(g)`

You'll adjust the grid length (`g`) depending on which orbital you're trying to plot. This will return three three-dimensional arrays x,y, z (as well as the grid spacing `h`).

The three-dimensional array x simply has the coordinate of x at every point in (the discretized) 3D space and the three-dimensional array y has the coordinate of y at every point in (the discretized) 3D space - e.g. the analogous 2D version of this are `x =`

```

[[-8. -4.  0.  4.  8.]
 [-8. -4.  0.  4.  8.]
 [-8. -4.  0.  4.  8.]
 [-8. -4.  0.  4.  8.]
 [-8. -4.  0.  4.  8.]]

```

and $y =$

```
[[-8. -8. -8. -8. -8.]
 [-4. -4. -4. -4. -4.]
 [ 0.  0.  0.  0.  0.]
 [ 4.  4.  4.  4.  4.]
 [ 8.  8.  8.  8.  8.]]
```

Python generates grids like this using `np.meshgrid` (see the `HydrogenGrid` function)

By using these three grids, it makes it easy to produce three-dimensional grids of the values of r or θ or ϕ .

Go ahead and write a function `Cartesian2Spherical(x,y,z)` which returns three-dimensional grids of r , θ , and ϕ at every point in space. Recall that

$$\theta = \tan^{-1}(y/x) + \pi$$

(use `np.arctan2(y,x)`)

and

$$\phi = \cos^{-1}(z/r)$$

You should now have access to r , θ , ϕ on your grid from which you can call both your radial function and spherical harmonics.

? Answer (start)

```
1 def HydrogenGrid(g=10, numGridPoints=50):
2     p = np.linspace(-g, g, numGridPoints)
3     h = p[1] - p[0]
4     x, y, z = np.meshgrid(p, p, p)
5     return x, y, z, h
6
7 def Cartesian2Spherical(x, y, z):
8     r_3d = np.sqrt(x**2 + y**2 + z**2)
9     r_3d = np.where(r_3d < 1e-10, 1e-10, r_3d)
10    theta = np.arctan2(y, x) # azimuthal angle
11    phi = np.arccos(np.clip(z / r_3d, -1.0, 1.0)) # polar angle
12    return r_3d, theta, phi
```

✓ Answer (end)

Now write a function `HydrogenAtom(n, l, m, x, y, z)` which computes the hydrogen orbital on this grid and returns `psi`.

The spherical Harmonics is

```
sp.sph_harm_y(l,m,phi,theta)
```

where the m and l are indexing the eigenstates of the angular momentum operator. (Notice that this is backwards from how you typical think about the quantum numbers for the hydrogen atom).

To plot the wave-function use the following

```
def PlotMe(psi,x,y,z):
    isosurface = go.Isosurface(
        x=x.flatten(),
        y=y.flatten(),
        z=z.flatten(),
        value=(np.real(psi)**2).flatten(), # Use the absolute value f
        surface_count=20, # Number of isosurfaces to display
        opacity=0.5, # Set the opacity of the isosurface
        surface_fill=1.0,
        caps=dict(x_show=False, y_show=False,z_show=False)
    )
    fig = go.Figure(data=[isosurface])
    fig.show()
```

Plot $(n, \ell, m) = (1, 0, 0); (2, 1, 0); (3, 2, 2); (3, 2, 0)$ where the grid-length l used to get a good figure was

```
g=dict()
g[(1,0,0)]=4
g[(2,1,0)]=7.7
g[(3,2,2)]=18
g[(3,2,0)]=20
```

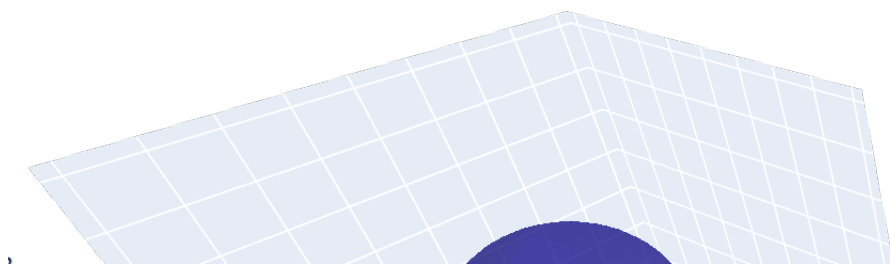
? Answer (start)

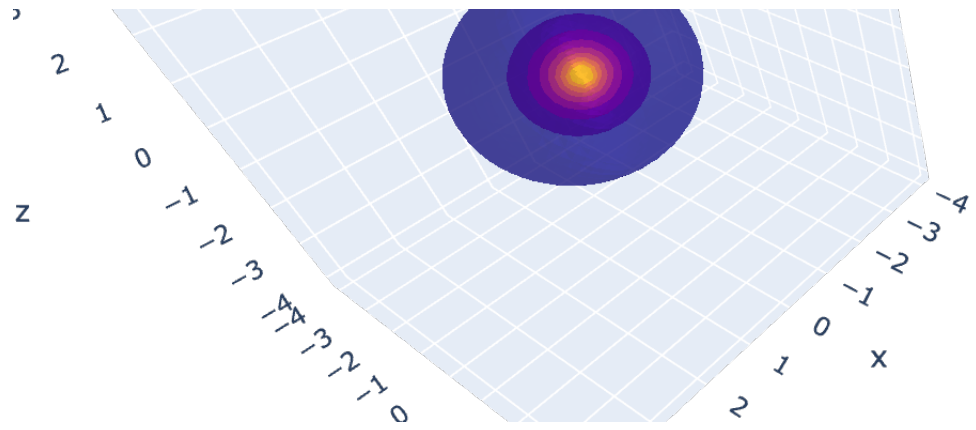
```

1 def HydrogenAtom(n, l, m, x, y, z):
2     """Full hydrogen wavefunction  $\Psi_{nlm}$  on a 3D Cartesian grid."""
3     r_grid, theta_grid, phi_grid = Cartesian2Spherical(x, y, z)
4     R = radial_function(n, l, r_grid)
5     Y = sp.sph_harm_y(l, m, phi_grid, theta_grid) # phi=polar,
6     return R * Y
7
8 def PlotMe(psi, x, y, z):
9     value_arr = (np.real(psi)**2).flatten()
10    isosurface = go.Isosurface(
11        x=x.flatten(), y=y.flatten(), z=z.flatten(),
12        value=value_arr,
13        isomin=0.02 * np.max(value_arr), # <-- add this
14        isomax=np.max(value_arr), # <-- and this
15        surface_count=10,
16        opacity=0.5, surface_fill=1.0,
17        caps=dict(x_show=False, y_show=False, z_show=False)
18    )
19    fig = go.Figure(data=[isosurface])
20    fig.show()
21
22 # Grid sizes chosen to capture each orbital well
23 g_vals = {(1,0,0): 4, (2,1,0): 7.7, (3,2,2): 18, (3,2,0): 20}
24
25 # Plot (1,0,0)
26 n, l, m = 1, 0, 0
27 x1, y1, z1, h1 = HydrogenGrid(g=g_vals[(n,l,m)], numGridPoints=5
28 psi_100 = HydrogenAtom(n, l, m, x1, y1, z1)
29 print(f"Plotting (n,l,m) = ({n},{l},{m})")
30 PlotMe(psi_100, x1, y1, z1)

```

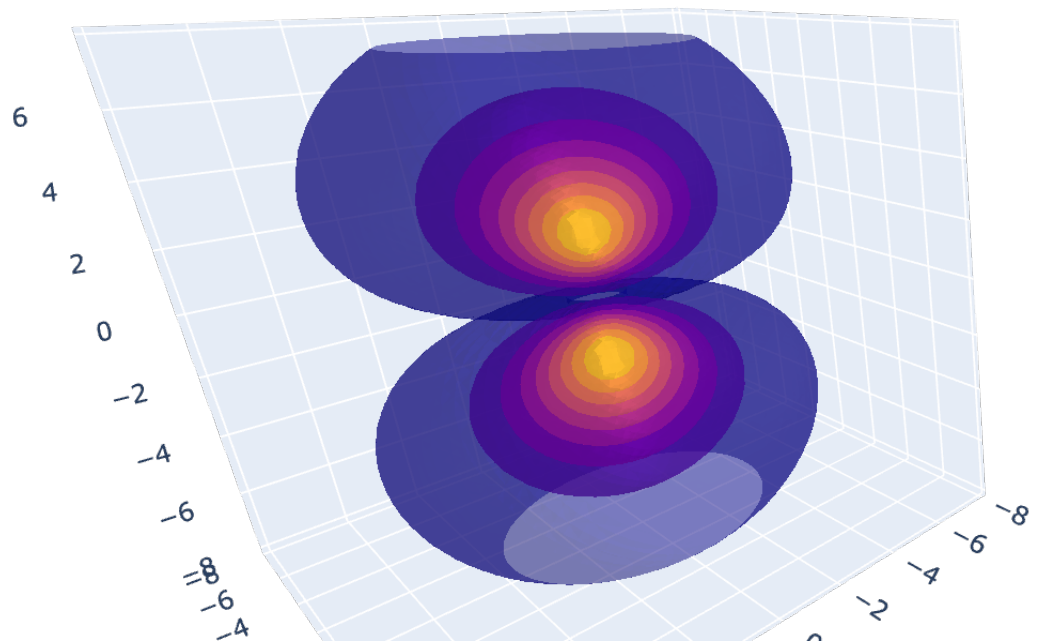
Plotting (n,l,m) = (1,0,0)





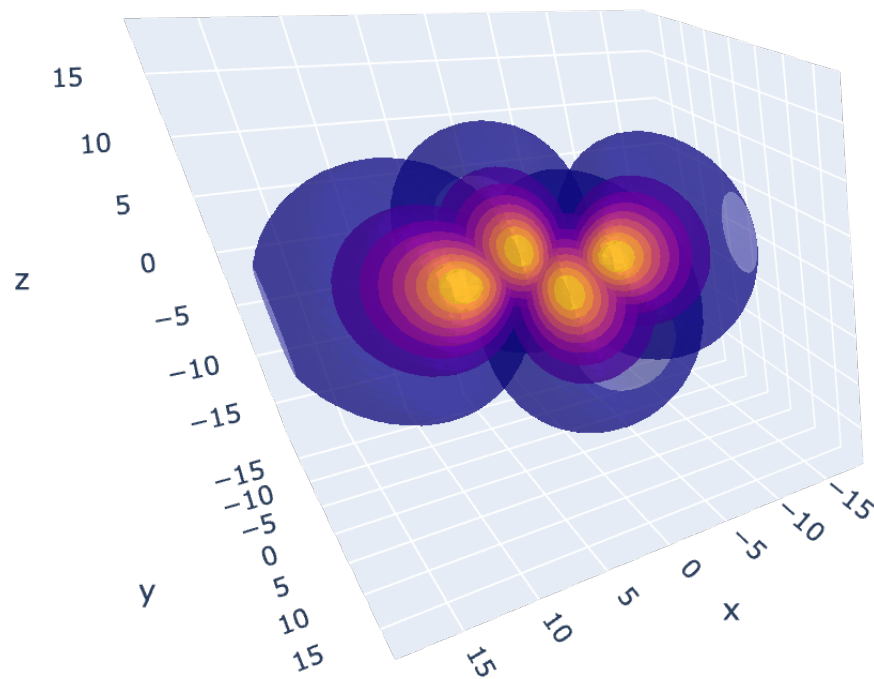
```
1 # Plot (2,1,0)
2 n, l, m = 2, 1, 0
3 x2, y2, z2, h2 = HydrogenGrid(g=g_vals[(n,l,m)], numGridPoints=5
4 psi_210 = HydrogenAtom(n, l, m, x2, y2, z2)
5 print(f"Plotting (n,l,m) = ({n},{l},{m})")
6 PlotMe(psi_210, x2, y2, z2)
```

Plotting (n,l,m) = (2,1,0)



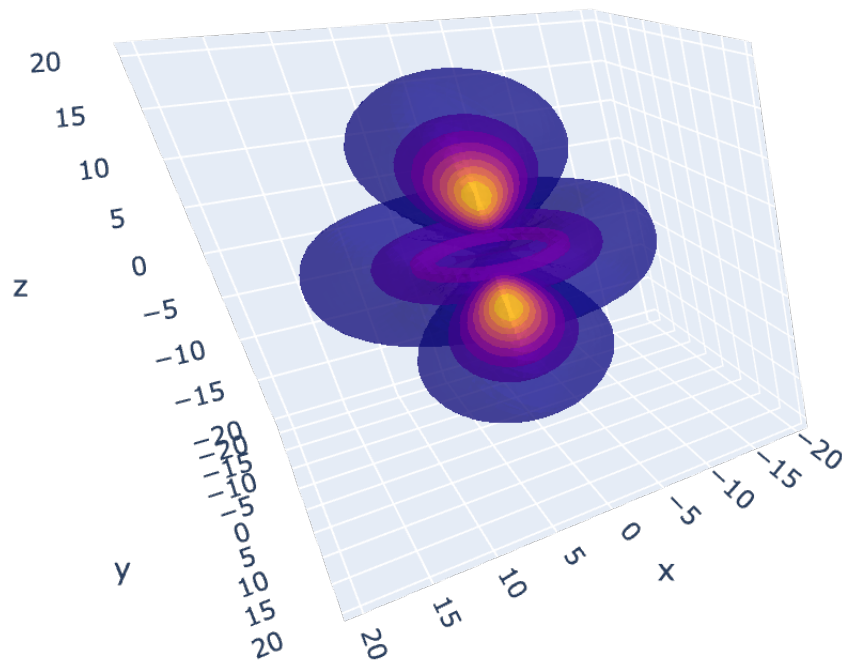
```
1 # Plot (3,2,2)
2 n, l, m = 3, 2, 2
3 x3a, y3a, z3a, h3a = HydrogenGrid(g=g_vals[(n,l,m)], numGridPoin
4 psi_322 = HydrogenAtom(n, l, m, x3a, y3a, z3a)
5 print(f"Plotting (n,l,m) = ({n},{l},{m})")
6 PlotMe(psi_322, x3a, y3a, z3a)
```

Plotting (n,l,m) = (3,2,2)



```
1 n, l, m = 3, 2, 0
2 x3b, y3b, z3b = HydrogenGrid(g=g_vals[(n,l,m)], numGridPoin
3 psi_320 = HydrogenAtom(n, l, m, x3b, y3b, z3b)
4 print(f"Plotting (n,l,m) = ({n},{l},{m})")
5 PlotMe(psi_320, x3b, y3b, z3b)
```

Plotting (n,l,m) = (3,2,0)



The 1s orbital is spherically symmetric; the 2p orbital shows dumbbell-shape with a nodal plane; the 3d orbitals (3,2,2) and (3,2,0) display the richer angular structure of d-type states, with larger spatial extent reflecting the higher principal quantum number.

✓ Answer (end)

✓ c. Getting the Energy of the Hydrogen Atom

Now we are going to compute the energy of the Hydrogen atom. Recall that the Hamiltonian for the Hydrogen atom is

$$H = -\frac{1}{2}\nabla^2 + \frac{1}{r}$$

where $-\nabla^2 = \partial^2/\partial x^2 + \partial^2/\partial y^2 + \partial^2/\partial z^2$ and r is the distance between the nucleus and the electron.

Let's figure out how to compute each of these terms.

This second term can be computed by taking the integral

$$\int \Psi(x, y, z) \left(-\frac{1}{r}\right) \Psi^*(x, y, z) dx dy dz$$

If we evaluate our wave-function `psi` on the grid (which is how we got our orbitals that we visualize above), then we simply multiply these three terms, sum them up (`np.sum`) and then multiply by our grid spacing cubed.

For the $(n, l, m) = (1, 0, 0)$ you should get a potential Energy of -0.9897509703021152 using the following grid

```
x, y, z, delta_x = HydrogenGrid(20, 200)
```

Implement this function and make sure you get the correct energy.

? Answer (start)

```
1 def potential_energy(n, l, m, x, y, z, h):
2     """⟨V⟩ = ∫ ψ* (-1/r) ψ d³r computed by direct grid quadrature
3     psi    = HydrogenAtom(n, l, m, x, y, z)
4     r_g    = np.sqrt(x**2 + y**2 + z**2)
5     r_g    = np.where(r_g < 1e-10, 1e-10, r_g)
6     return float(np.real(np.sum(np.conj(psi) * (-1.0 / r_g) * p
7
8 x_e, y_e, z_e, delta_x = HydrogenGrid(20, 200)
9 PE_100 = potential_energy(1, 0, 0, x_e, y_e, z_e, delta_x)
10 print(f"PE for (1,0,0) = {PE_100:.10f}")
11 print(f"Expected          = -0.9897509703")
```

```
PE for (1,0,0) = -0.9897509703
Expected       = -0.9897509703
```

✓ Answer (end)

Now we need to get the next term

$$-\frac{1}{2} \int \Psi^*(x, y, z) \frac{\partial^2}{\partial x^2} \Psi(x, y, z) dx dy dz$$

We can compute this by finite differences. The standard formula for the second derivative is

$$\frac{\partial^2}{\partial x^2} = \frac{f(x + dx) + f(x - dx) - 2f(x)}{dx^2}$$

Write a function `Laplacian(n, l, m)` which calls `HydrogenWaveFunction` three times (per direction) returning the laplacian; for example for the x-direction it calls `x+dx`, `x-dx`, and `x`. For `dx` use `1e-3`.

Compute the kinetic energy of the $(n, l, m) = (1, 0, 0)$ state. You should get 0.4897835961971023 total with each of the three kinetic terms being the same (because of the symmetry of this state)

Now sum up the kinetic and potential energies and multiply by 27.2 to get it into eV. Verify that you get the right answer for the (1,0,0) term. Also compute the energies for the (2,0,0), (2,1,0), (2,1,1) and (3,2,1) terms. Check that each of them are close to the known value.

? Answer (start)

```

1 def Laplacian(n, l, m, x, y, z, dx=1e-3):
2     """Finite-difference Laplacian of  $\Psi_{nlm}$ ; returns ( $d^2x$ ,  $d^2y$ ,
3     psi    = HydrogenAtom(n, l, m, x,      y,      z      )
4     psi_xp = HydrogenAtom(n, l, m, x + dx, y,      z      )
5     psi_xm = HydrogenAtom(n, l, m, x - dx, y,      z      )
6     psi_yp = HydrogenAtom(n, l, m, x,      y + dx, z      )
7     psi_ym = HydrogenAtom(n, l, m, x,      y - dx, z      )
8     psi_zp = HydrogenAtom(n, l, m, x,      y,      z + dx)
9     psi_zm = HydrogenAtom(n, l, m, x,      y,      z - dx)
10    d2x = (psi_xp + psi_xm - 2 * psi) / dx**2
11    d2y = (psi_yp + psi_ym - 2 * psi) / dx**2
12    d2z = (psi_zp + psi_zm - 2 * psi) / dx**2
13    return d2x, d2y, d2z, psi
14
15 x_e, y_e, z_e, h_e = HydrogenGrid(20, 200)
16 d2x, d2y, d2z, psi0 = Laplacian(1, 0, 0, x_e, y_e, z_e)
17 h3 = h_e**3
18 KE_x = float(np.real(np.sum(np.conj(psi0) * (-0.5) * d2x)) * h3
19 KE_y = float(np.real(np.sum(np.conj(psi0) * (-0.5) * d2y)) * h3
20 KE_z = float(np.real(np.sum(np.conj(psi0) * (-0.5) * d2z)) * h3
21 KE_total = KE_x + KE_y + KE_z
22 print(f"KE_x = {KE_x:.10f}, KE_y = {KE_y:.10f}, KE_z = {KE_z:.10f}")
23 print(f"Total KE = {KE_total:.10f} (expected  $\approx 0.4897835961$ )")
24
25 PE_100 = potential_energy(1, 0, 0, x_e, y_e, z_e, h_e)
26 E_100 = KE_total + PE_100
27 print(f"\nTotal E (1,0,0) = {E_100:.6f} a.u. = {E_100 * 27.2:.4f} eV")
28
29 print("\n (n,l,m)  E (a.u.)  E (eV)  exact (eV)")
30 for ns, ls, ms in [(1,0,0),(2,0,0),(2,1,0),(2,1,1),(3,2,1)]:
31     d2x_s, d2y_s, d2z_s, psi_s = Laplacian(ns, ls, ms, x_e, y_e, z_e, h_e)
32     KE_s = float(np.real(np.sum(np.conj(psi_s)*(-0.5)*(d2x_s+d2y_s+d2z_s)) * h3)
33     PE_s = potential_energy(ns, ls, ms, x_e, y_e, z_e, h_e)
34     E_s = KE_s + PE_s
35     print(f" ({ns},{ls},{ms})  {E_s:+.5f}  {E_s*27.2:+.4f} eV")

```

KE_x = 0.1632611987, KE_y = 0.1632611987, KE_z = 0.1632611987
Total KE = 0.4897835962 (expected ≈ 0.4897835961)

Total E (1,0,0) = -0.499967 a.u. = -13.5991 eV (exact: -13.6 eV)

(n,l,m)	E (a.u.)	E (eV)	exact (eV)
(1,0,0)	-0.49997	-13.5991	-13.6000
(2,0,0)	-0.12500	-3.3999	-3.4000
(2,1,0)	-0.12500	-3.4000	-3.4000
(2,1,1)	-0.12500	-3.4000	-3.4000
(3,2,1)	-0.05536	-1.5058	-1.5111

✓ Answer (end)

✓ Exercise 2. Solving the Hydrogen Atom Radial Equation Numerically

Recall that the Hydrogen atom Hamiltonian is

$$H = -\frac{1}{2}\nabla^2 - \frac{1}{r}$$

We solve the hydrogen atom using separation of variables eventually finding that

$$\Psi(R, \theta, \phi) = R_{nl}(r)Y_{lm}(\theta, \phi)$$

where $R_{nl}(r) = u(r)/r$ and $u(r)$ obeys the radial eigenvalue equation

$$\left[-\frac{1}{2} \left(\frac{d^2}{dr^2} - \frac{l(l+1)}{r^2} \right) - \frac{1}{r} \right] u(r) = Eu(r)$$

which we can write as

$$H_{eff}u(r) = Eu(r)$$

This problem looks a lot like the particle in the box. We can think of r here just like the x coordinate. Then the Hamiltonian has three terms:

- the standard kinetic energy term
- a potential term coming from the angular momentum
- a potential term coming from the coulomb potential.

To solve this analytically required a significant amount of work. We will see that solving it numerically is relatively straightforward. For each of the terms in H_{eff} we need to write out a $N \times N$ Hamiltonian matrix (where N is the number of points in our grid which discretizes space).

✓ a. The Kinetic Term

We will start by working with a discretized grid which is $N = 200$ points defined by

```
N=200
r = np.linspace(40, 0.0, N, endpoint=False)
```

Later to actually get accurate answers, we are going to want to increase N which will actually require us to make some changes (moving to sparse matrices) but let's start with the smaller dense matrices.

We are building $N \times N$ matrices where each row (respectively column) corresponds to a values of r on the grid - i.e. the "third" row corresponds to $r[2]$ and the "fourth" column corresponds to the value of $r[3]$.

Let's start with the term

$$\frac{d^2}{dr^2} = \frac{2f(r) - f(r + dr) - f(r - dr)}{dr^2}$$

As a matrix this means:

- the elements that connect the r 'th column to the r 'th row should get a $2/dr^2$. This is just the diagonal of the matrix
- the elements that connect the $(r+1)$ 'st column to the r 'th row should get a $-1/dr^2$. This is just one row above the diagonal
- the elements that connect the $(r-1)$ 'st column to the r 'th row should get a $-1/dr^2$. This is just one row below the diagonal.

Write a function `Laplacian(r)` which takes the grid `r` and returns a matrix corresponding to the second derivative. You will find the `np.diag` function useful (even the off-diagonal term can be built using it - look at the documentation!). You should essentially be able to build this matrix in three such function calls.

Working in units where $\hbar = 1$ and $m = 1$, go ahead and compute the full kinetic term. To diagonalize a matrix, you can call `e, v=np.linalg.eigh(-0.5*L)`. Verify that the lowest four energies are `[0.00305358 0.01221356 0.02747771 0.04884231]`.

? Answer (start)

```

1 N = 200
2 r = np.linspace(40, 0.0, N, endpoint=False)
3 dr = abs(r[1] - r[0])
4
5 def Laplacian(r_grid):
6     """1-D second-derivative matrix d2/dr2 (standard form: nega
7     N_r = len(r_grid)
8     dr_r = abs(r_grid[1] - r_grid[0])
9     L = (np.diag(-2.0 * np.ones(N_r) / dr_r**2) +
10         np.diag( 1.0 * np.ones(N_r-1) / dr_r**2, k= 1) +
11         np.diag( 1.0 * np.ones(N_r-1) / dr_r**2, k=-1))
12     return L
13
14 L = Laplacian(r)
15 e_kin, _ = np.linalg.eigh(-0.5 * L)
16 print(f"Lowest 4 kinetic energies: {e_kin[:4]}")
17 print(f"Expected:                [0.00305358 0.01221356 0.02747771 0.04884

```

```

Lowest 4 kinetic energies: [0.00305358 0.01221356 0.02747771 0.04884
Expected:                [0.00305358 0.01221356 0.02747771 0.04884

```

✓ Answer (end)

✓ b. The potential terms

The two potential terms are a function of r and their matrices for these two terms therefore only connect the r 'th row to the r 'th column. This means both additional terms only exist on the diagonal. Build a function $V(r, l)$ that takes the r grid and returns the matrix corresponding to the potential

$$V(r, l) = \frac{l(l+1)}{2r^2} - \frac{1}{r}$$

Again use `np.diag`

? Answer (start)

```

1 def V(r_grid, l_val):
2     """Effective radial potential: centrifugal barrier + Coulom
3     return np.diag(l_val * (l_val + 1) / (2.0 * r_grid**2) - 1.

```

✓ Answer (end)

✓ c. Putting it together

Now we simply want to add the full Hamiltonian matrix together and diagonalize it using `e,v=np.linalg.eigh(H)`

Both print and plot the lowest 6 energies - `plt.plot(e[0:6]*27.2)`. Also on your plot make some straight lines - `plt.axhline(the_energy)` where the energies for the n'th eigenstate should be - i.e. $-13.6/n^2$. How close are you getting?

Also plot the lowest three eigenstates and see if the radial functions match the ones from above. To normalize the plot correctly you have to divide your eigenvectors by $\sqrt{h}$.

? Answer (start)

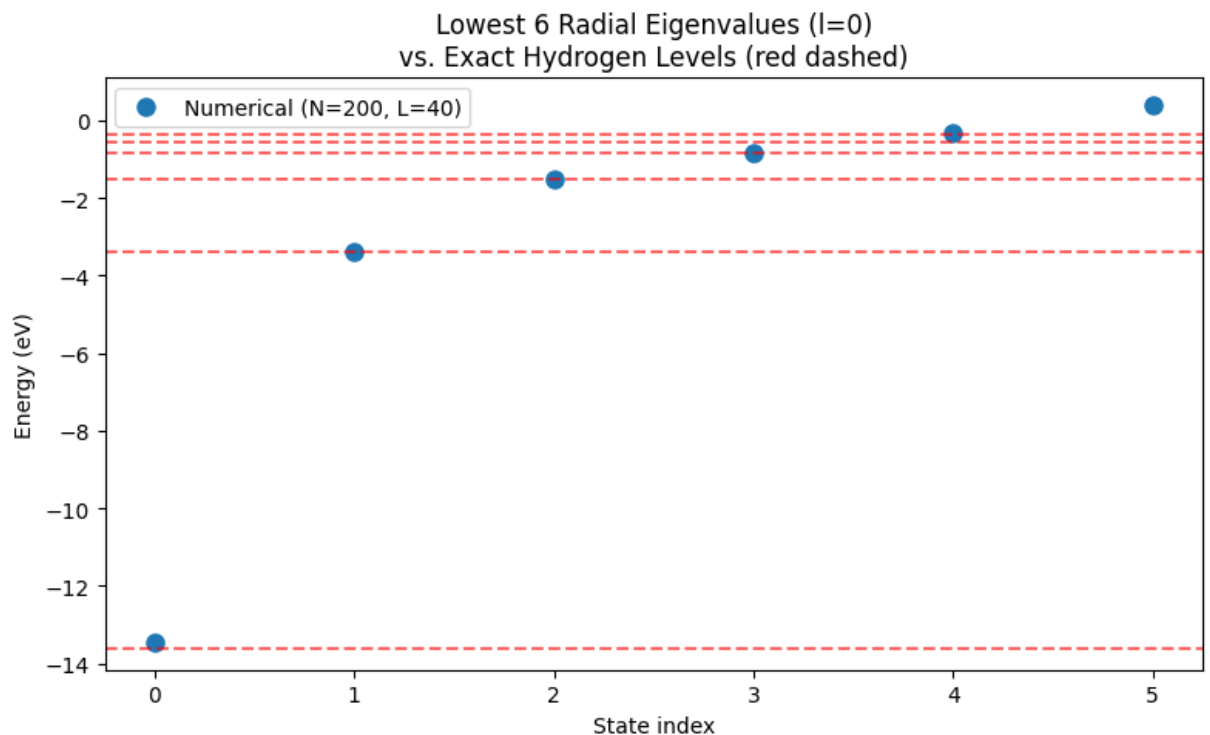
```

1 l_val = 0
2 H_rad = -0.5 * Laplacian(r) + V(r, l_val)
3 e_rad, v_rad = np.linalg.eigh(H_rad)
4
5 print("Lowest 6 energies (eV):", e_rad[:6] * 27.2)
6 print("Exact -13.6/n^2 (eV):  ", [-13.6 / n**2 for n in range(1
7
8 plt.figure(figsize=(8, 5))
9 plt.plot(range(6), e_rad[:6] * 27.2, 'o', ms=8, label='Numerica
10 for n in range(1, 7):
11     plt.axhline(-13.6 / n**2, color='red', linestyle='--', alph
12 plt.xlabel('State index')
13 plt.ylabel('Energy (eV)')
14 plt.title('Lowest 6 Radial Eigenvalues (l=0)\nvs. Exact Hydroge
15 plt.legend()
16 plt.tight_layout()
17 plt.show()

```

Lowest 6 energies (eV): [-13.46665385 -3.39154224 -1.50940275 -0.41147282]

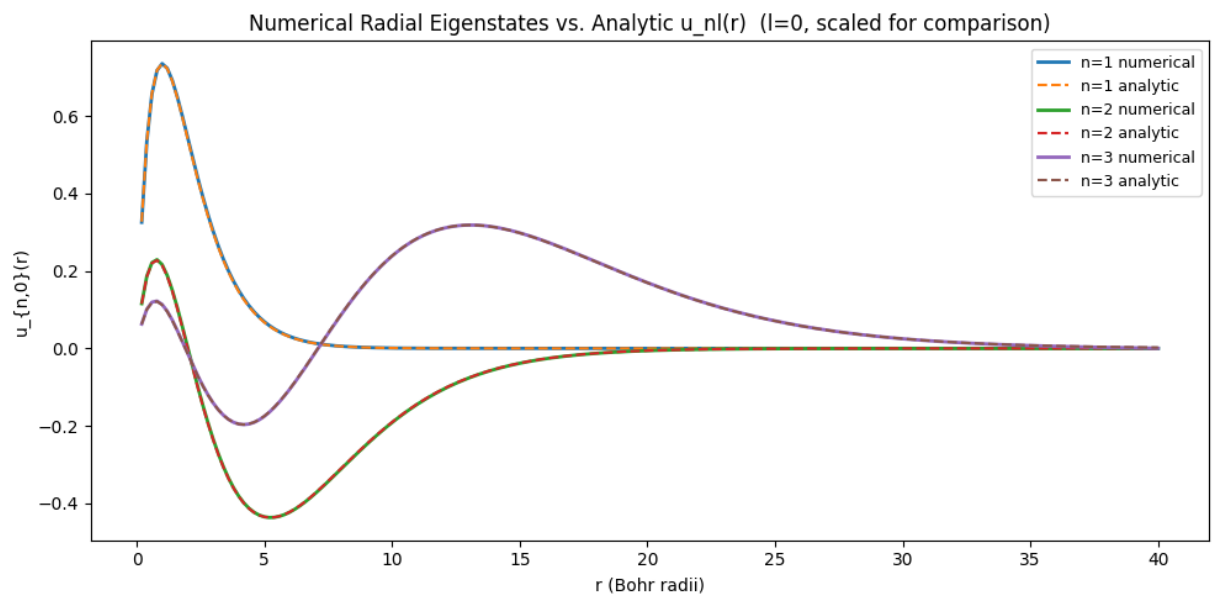
Exact $-13.6/n^2$ (eV): [-13.6, -3.4, -1.5111111111111111, -0.85, -0.41147282]



```

1 sort_idx = np.argsort(r)
2 r_sorted = r[sort_idx]
3
4 plt.figure(figsize=(10, 5))
5 for i in range(3):
6     n_val = i + 1
7     psi_num = v_rad[:, i] / np.sqrt(dr)      # normalize by sqrt
8     # align sign with analytic convention (1s is positive near
9     u_ana = r_sorted * radial_function(n_val, 0, r_sorted)
10    if psi_num[sort_idx[-1]] * u_ana[-1] < 0:
11        psi_num = -psi_num
12    scale = np.max(np.abs(u_ana)) / np.max(np.abs(psi_num))
13    plt.plot(r[sort_idx], psi_num[sort_idx] * scale, label=f'n=
14    plt.plot(r_sorted, u_ana, '--', label=f'n=
15
16 plt.xlabel('r (Bohr radii)')
17 plt.ylabel('u_{n,0}(r)')
18 plt.title('Numerical Radial Eigenstates vs. Analytic u_nl(r) (
19 plt.legend(fontsize=9)
20 plt.tight_layout()
21 plt.show()
22

```



The numerical eigenstates match the analytic $u_{nl}(r)$ closely for the lowest n ; higher- n states have longer spatial extent and require a larger grid for full accuracy.

✓ Answer (end)

✓ d. Finer Grids

We are not getting an accuracy as high as we might desire. What we need is a better grid: finer and going out to larger r . Unfortunately, if there are more grid points, the matrix gets larger and using dense matrices and solving for all the eigenstates becomes intractable. Instead we need to work with sparse matrices.

To do this, you need to do the following:

- Change `V` and `Laplacian` to return sparse matrices. You can do this by just having the return be `return scipy.sparse.csr_matrix(oldMatrix)`
- When you diagonalize, take `e,v=scipy.sparse.linalg.eigsh(H,k=6,which='SA')` This will give you sparse versions of the lowest six eigenstates.

Use now a grid of $N = 4000$ out to $L = 60$

Go ahead and plot the same curves as above and verify you get more reasonable results.

? Answer (start)

```
1 # Override Laplacian and V to return sparse matrices for the fi
2 def Laplacian(r_grid):
3     N_r = len(r_grid)
4     dr_r = abs(r_grid[1] - r_grid[0])
5     return scipy.sparse.diags(
6         [-2.0 * np.ones(N_r) / dr_r**2,
7          1.0 * np.ones(N_r-1) / dr_r**2,
8          1.0 * np.ones(N_r-1) / dr_r**2],
9         [0, 1, -1], format='csr')
```

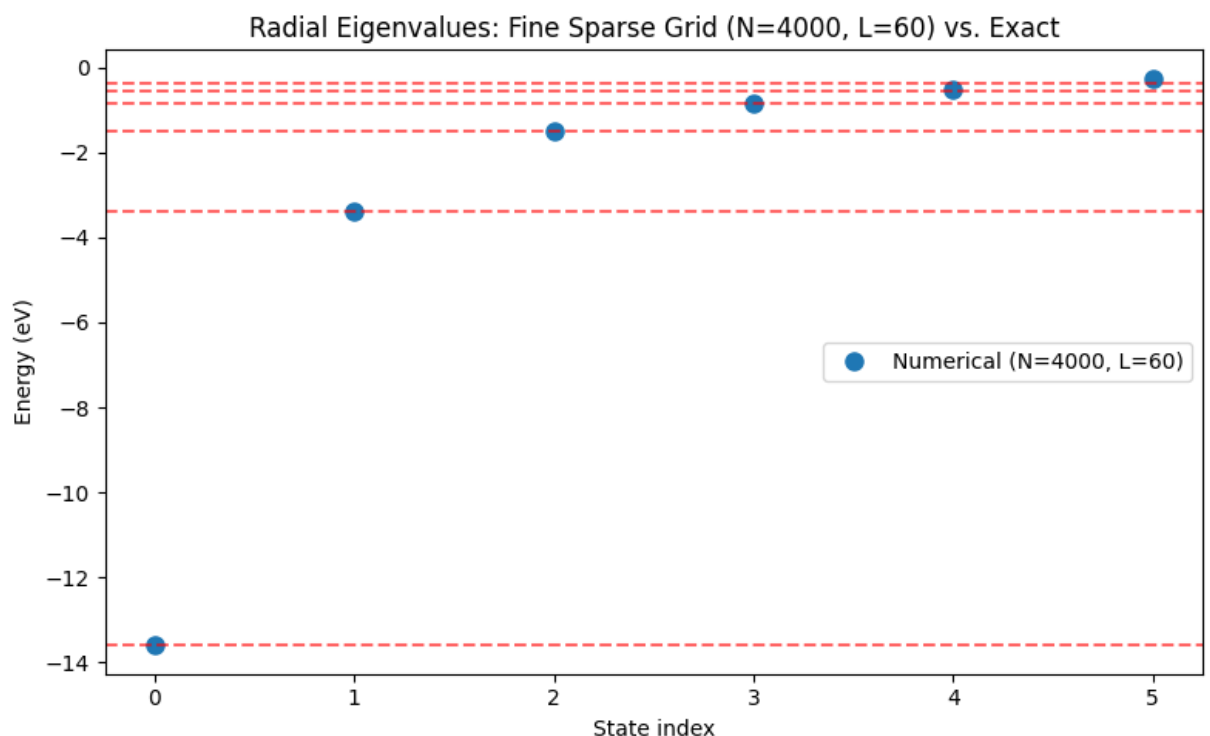
```

10
11 def V(r_grid, l_val):
12     diag_vals = l_val * (l_val + 1) / (2.0 * r_grid**2) - 1.0 /
13     return scipy.sparse.diags(diag_vals, format='csr')
14
15 N_fine = 4000
16 r_fine = np.linspace(60, 0.0, N_fine, endpoint=False)
17
18 H_sp = -0.5 * Laplacian(r_fine) + V(r_fine, 0)
19 e_sp, _ = scipy.sparse.linalg.eigsh(H_sp, k=6, which='SA')
20 e_sp_sorted = np.sort(e_sp)
21
22 print("Fine grid (N=4000, L=60) lowest 6 energies (eV):", e_sp_
23 print("Exact  $-13.6/n^2$  (eV):", [-13.
24
25 plt.figure(figsize=(8, 5))
26 plt.plot(range(6), e_sp_sorted * 27.2, 'o', ms=8, label='Numeri
27 for n in range(1, 7):
28     plt.axhline(-13.6 / n**2, color='red', linestyle='--', alph
29 plt.xlabel('State index')
30 plt.ylabel('Energy (eV)')
31 plt.title('Radial Eigenvalues: Fine Sparse Grid (N=4000, L=60)
32 plt.legend()
33 plt.tight_layout()
34 plt.show()
35

```

Fine grid (N=4000, L=60) lowest 6 energies (eV): [-13.59923509 -3.3
-0.25801611]

Exact $-13.6/n^2$ (eV): [-13.6, -3.4, -1.51



The sparse fine grid (N=4000, L=60) produces eigenvalues that are dramatically closer to the exact $-13.6/n^2$ values, confirming that higher resolution and a larger domain systematically improve the accuracy of the numerical diagonalization.

✓ Answer (end)

✓ Exercise 3. Hydrogen Atom on a Grid

So far we've plotted the orbitals for the Hydrogen atom by

- using the formula for the orbitals that one derives analytically
- using the radial equation you derived analytically (along with the spherical harmonics)

In both cases, we (somebody?) had to do a lot of analytic work to get something that we then worked with computationally. Now we just want to envision a situation where we have a Hamiltonian and want to directly work with and diagonalize this on a grid.

✓ a. The Hamiltonian

Our goal here is to be able to solve for the eigenstates and eigenvalues of the Hydrogen atom.

The hydrogen atom has a single electron and a Hamiltonian of

$$H = -\frac{1}{2}\nabla^2 + V(R)$$

where

$$V(R) = -\frac{1}{R}$$

where R is the distance between the proton and electron of the Hydrogen atom.

Let's learn some things to put this together:

First recall that $\nabla^2 = \frac{d^2}{dx^2} + \frac{d^2}{dy^2} + \frac{d^2}{dz^2}$. We need to generate a matrix for this laplacian.

There's something a little subtle here. d^2/dx^2 looks like the same term that we saw for the particle in the box but it now needs to exist over three-dimensions. If L was the one-dimensional derivative, we instead now write this as $L \otimes I \otimes I$ where I is the identity of the same size matrix.

Let's think how large a matrix we are making here. We have a matrix that is number of grid points $\times$ grid points. If we use 80 grid points in every direction, we get a matrix of size $80^3 \times 80^3$ or $512,000 \times 512,000$. This is much too large to be densely represented on a computer. To get around this, we are going to have to use sparse matrices. The trick here is to

- first take your one-dimensional Laplacian (call it L) and turn it into a sparse matrix (i.e. `scipy.sparse.lil_matrix(L)`) and
- then when you produce your three-dimensional laplacian make sure that you do a kroneker product - i.e.

`scipy.sparse.kron(scipy.sparse.kron(L,I),I)` These two steps will give you $\frac{d^2}{dx^2}$. Then you need to also get $\frac{d^2}{dy^2}$ and $\frac{d^2}{dz^2}$. Go ahead and generate the matrix then for ∇^2 (make sure in your laplacian you remember to divide by $1/h^2$ where h is the grid spacing).

Let's check that our Laplacian is giving us the correct thing. We should be able to diagonalize it and get the energies for a particle in a three-dimensional box.

You can't actually afford to get all the eigenvalues of such a large matrix. Instead, we are going to only get the first couple. To do this, you can use

`e,v=scipy.sparse.linalg.eigsh(-0.5*L, which='SA')` and show that the first few energies are correct for a grid

```
HydrogenGrid(12,numGridPoints=80)
```

The `which='SA'` is important to get the lowest energies.

? Answer (start)

```

1 def Laplacian(h, g):
2     """3-D Laplacian  $\nabla^2$  as a sparse matrix via Kronecker product
3     h = grid spacing, g = number of points per dimension.
4     State vector is indexed as psi[iy*g2+ix*g+iz] (C/row-major)
5     """
6     L1 = scipy.sparse.diags(
7         [-2.0 * np.ones(g) / h**2,
8          1.0 * np.ones(g-1) / h**2,
9          1.0 * np.ones(g-1) / h**2],
10        [0, 1, -1], format='lil')
11     I = scipy.sparse.eye(g, format='lil')
12
13     #  $\nabla^2 = d^2/dy^2 \otimes I \otimes I + I \otimes d^2/dx^2 \otimes I + I \otimes I \otimes d^2/dz^2$ 
14     Ly = scipy.sparse.kron(scipy.sparse.kron(L1, I), I)
15     Lx = scipy.sparse.kron(scipy.sparse.kron(I, L1), I)
16     Lz = scipy.sparse.kron(scipy.sparse.kron(I, I), L1)
17     return scipy.sparse.csr_matrix(Lx + Ly + Lz)
18
19 # --- Sanity check: particle in a 3-D box ---
20 x_box, y_box, z_box, h_box = HydrogenGrid(g=12, numGridPoints=8
21 g_pts = 80
22 L3D_box = Laplacian(h_box, g_pts)
23
24 print("Testing 3-D Laplacian with particle in a box (L=24, g=80) ...")
25 e_box, _ = scipy.sparse.linalg.eigsh(-0.5 * L3D_box, k=6, which='sm')
26 e_box = np.sort(e_box)
27 print(f"Numerical lowest 3 kinetic energies: {e_box[:3]}")
28
29 # Exact 3-D particle-in-box energies:  $E = \pi^2(n_1^2 + n_2^2 + n_3^2) / (2 * L_{box}^2)$ 
30 L_box = 24.0
31 E_pib = lambda n1, n2, n3: np.pi**2 * (n1**2 + n2**2 + n3**2) / (2 * L_box**2)
32 print(f"Expected (1,1,1) = {E_pib(1,1,1):.5f}")
33 print(f"Expected (1,1,2) = {E_pib(1,1,2):.5f}")

```

```

Testing 3-D Laplacian with particle in a box (L=24, g=80) ...
Numerical lowest 3 kinetic energies: [0.02444546 0.04887866 0.04887866]
Expected (1,1,1) = 0.02570
Expected (1,1,2) = 0.05140

```

✓ Answer (end)

After you have your laplacian in place, you want to generate the Coulomb potential piece of your Hamiltonian ($1/R$) on a grid.

The potential $V(R)$ is a diagonal term of the Hamiltonian (i.e. it's on the diagonal of the matrix). First compute the potential $V(R)$ on your three-dimensional grid (call it `Vext`). To put it on the diagonal, you need to reshape your three-dimensional grid onto a one-dimensional vector which you can then make the diagonal of your matrix

```
m=np.shape(Vext)[0]
Vext=np.reshape(Vext,m*m*m)
potential=scipy.sparse.diags(Vext)
```

Write a function `ExternalPotential` which generates this matrix.

You now have your laplacian term and your potential term. You can add these two terms together to get your full Hamiltonian.

? Answer (start)

```
1 def ExternalPotential(x, y, z):
2     """Coulomb potential V(R) = -1/R as a sparse diagonal matrix
3     R      = np.sqrt(x**2 + y**2 + z**2)
4     R      = np.where(R < 1e-10, 1e-10, R)
5     Vext = -1.0 / R
6     m     = np.shape(Vext)[0]
7     Vext = np.reshape(Vext, m * m * m)
8     return scipy.sparse.diags(Vext, format='csr')
9
10 # Build the full 3-D hydrogen Hamiltonian H = +0.5 P2 + V(R)
11 x_H, y_H, z_H, h_H = HydrogenGrid(g=12, numGridPoints=80)
12 g_H = 80
13 L3D = Laplacian(h_H, g_H)
14 V_H = ExternalPotential(x_H, y_H, z_H)
15 H_3D = -0.5 * L3D + V_H          # kinetic (-1/2 ∇²) + Coulomb (-1
16
17 print(f"Full 3-D Hamiltonian built: shape = {H_3D.shape}")
```

```
Full 3-D Hamiltonian built: shape = (512000, 512000)
```

✓ Answer (end)

✓ b. Eigenvalues

Diagonalize your full Hamiltonian using `scipy.sparse.linalg.eigsh` on your $12 \times 12 \times 12$ grid.

We would like to see that you get the expected eigenenergies and eigenstates.

Recall the standard hydrogen orbitals from chemistry each with an energy of $-13.6/n^2$ eV

The lowest 6 energies for the Hydrogen atom should be

- 1s
- 2s
- 2px, 2py, 2pz
- 3s

Let's start with looking at the energies of the 1s, 2s, and 3s states which we expect it to be -13.6 eV, -13.6/4 eV and -13.6/9 eV respectively. Obviously because we are using a grid, we aren't going to get the exact answer. But as we increase the grid size we should get closer and closer to the true answer. Starting with 20 grid points per direction and increasing the grid in units of 20 up to 100 grid points per direction, graph the energy of these three eigenstates as a function of number of grid points per direction. Also draw horizontal lines at the expected value of the eigenstates. You should see the energies approach the target energies at large grid-size. *Hint: Your $g=20$ energy for 1s should be -10.71473301.* (There are some eigenstates where you need to increase the size of the grid to capture the full wave-function)

Note:. You need to be careful about units. To convert into eV you have to multiply your energy by 27.21.

? Answer (start)

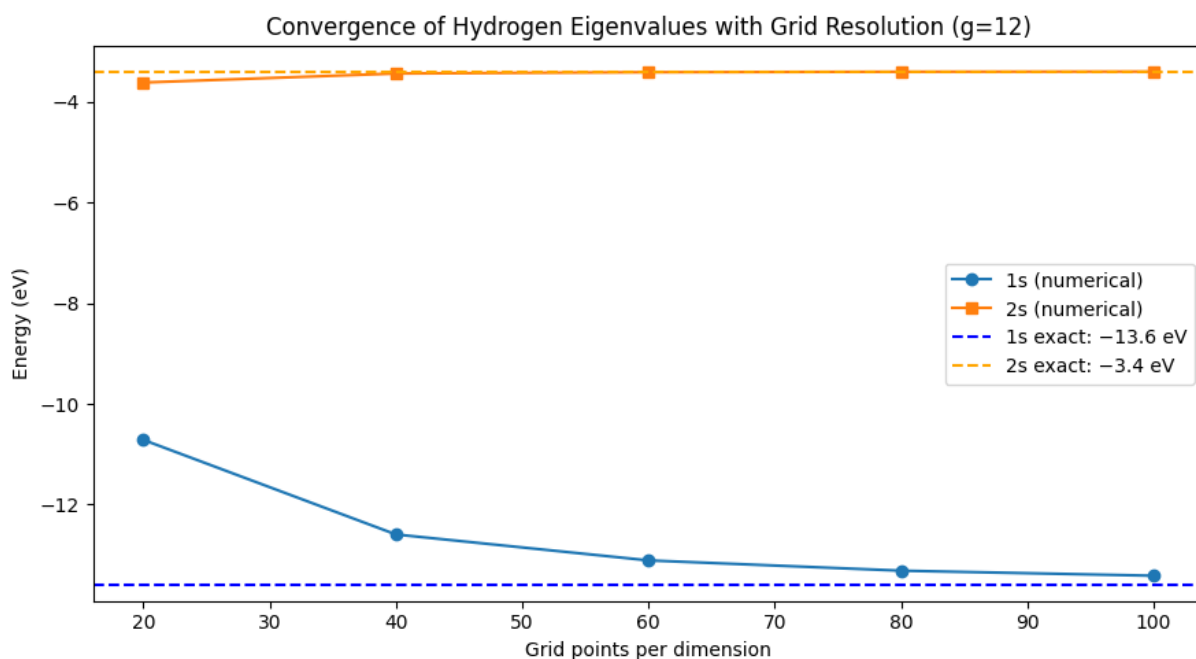
```
1 # Convergence study: vary numGridPoints from 20 to 100 in steps
2 grid_points_list = [20, 40, 60, 80, 100]
3 E_1s_list, E_2s_list = [], []
4
5 for gp in grid_points_list:
6     xg, yg, zg, hg = HydrogenGrid(g=12, numGridPoints=gp)
7     Lg = Laplacian(hg, gp)
8     Vg = ExternalPotential(xg, yg, zg)
9     Hg = -0.5 * Lg + Vg
10    eg, _ = scipy.sparse.linalg.eigsh(Hg, k=6, which='SA')
11    eg_eV = np.sort(eg) * 27.21
12    E_1s_list.append(eg_eV[0])
13    E_2s_list.append(eg_eV[1])
14    print(f"gp={gp:3d}: E_1s = {eg_eV[0]:8.5f} eV, E_2s = {eg_eV[1]:8.5f} eV")
```

```
gp= 20: E_1s = -10.71473 eV, E_2s = -3.61156 eV
gp= 40: E_1s = -12.59472 eV, E_2s = -3.43223 eV
gp= 60: E_1s = -13.11034 eV, E_2s = -3.40625 eV
gp= 80: E_1s = -13.31464 eV, E_2s = -3.39741 eV
gp=100: E_1s = -13.41480 eV, E_2s = -3.39326 eV
```

```

1 plt.figure(figsize=(9, 5))
2 plt.plot(grid_points_list, E_1s_list, 'o-', label='1s (numerical)')
3 plt.plot(grid_points_list, E_2s_list, 's-', label='2s (numerical)')
4 plt.axhline(-13.6, color='blue', linestyle='--', label='1s exact')
5 plt.axhline(-13.6/4, color='orange', linestyle='--', label='2s exact')
6 plt.xlabel('Grid points per dimension')
7 plt.ylabel('Energy (eV)')
8 plt.title('Convergence of Hydrogen Eigenvalues with Grid Resolution')
9 plt.legend()
10 plt.tight_layout()
11 plt.show()
12

```



The 3-D isosurfaces confirm the spherical shell of the 1s orbital and the two lobes separated by a nodal plane of the 2p orbital, in excellent qualitative agreement with the analytically derived orbitals from Exercise 1.

✓ Answer (end)

✓ c. Eigenvectors

Now we can also look at the eigenstates. For this section, let's use a grid with 80 points.

Let's start by looking at the s-states. First, we will make some two-dimensional slices of the three-dimensional wave-functions we've found. To get your 2d visualization, you first need to reshape your wave-function into a 3d grid - if you have a $80 \times 80 \times 80$ grid this can be done for the i th eigenstate by doing `A=psi[:,i].reshape(80,80,80)`. Now you can go ahead and plot it by `plt.matshow(A[40,:,:])` where 40 is the slice that you want. You probably want to use `plt.colorbar()` so you can see the magnitude and sign of things.

If you look at the 0 (1s), 1 (2s) or 5 (3s) eigenstate you will find that it should have a clear "S" character - symmetric around the origin. (This ordering of eigenstates)

You can also graph the probability density as a function of the radius. You could average but because it is symmetric the easiest thing to do is just plot a one dimensional line starting at the origin. To match up with earlier results, actually we'd like to plot not the radial function $|R_{nl}(r)|^2$ but $|u_{nl}(r)|^2$ so multiply by r - i.e. `plt.plot(r, r*A[40,40,40:])`. Notice we are starting at `40:` for that last term because that's where zero starts. For the different shells, count how many nodes you see (i.e. places where the probability density goes to zero). To normalize you have to divide by $1/(4\pi\sqrt{h})$

? Answer (start)

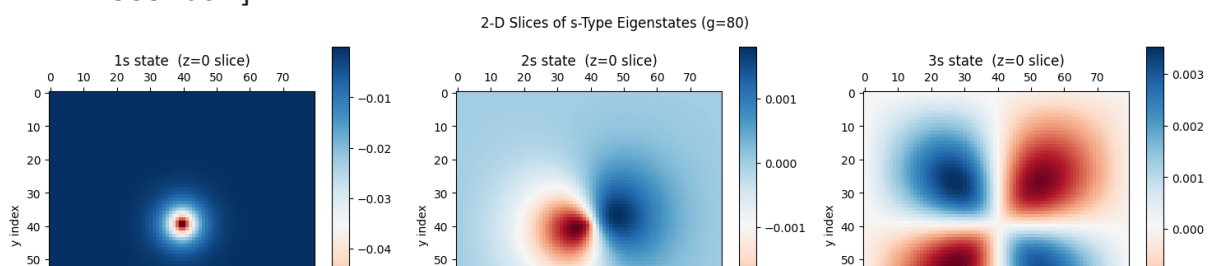
```
1 # Solve with g=80 for good accuracy (re-use H_3D from cell above)
2 # otherwise rebuild here)
3 try:
4     H_3D
5 except NameError:
6     x_H, y_H, z_H, h_H = HydrogenGrid(g=12, numGridPoints=80)
7     g_H = 80
8     L3D = Laplacian(h_H, g_H)
9     V_H = ExternalPotential(x_H, y_H, z_H)
10    H_3D = -0.5 * L3D + V_H
11
```

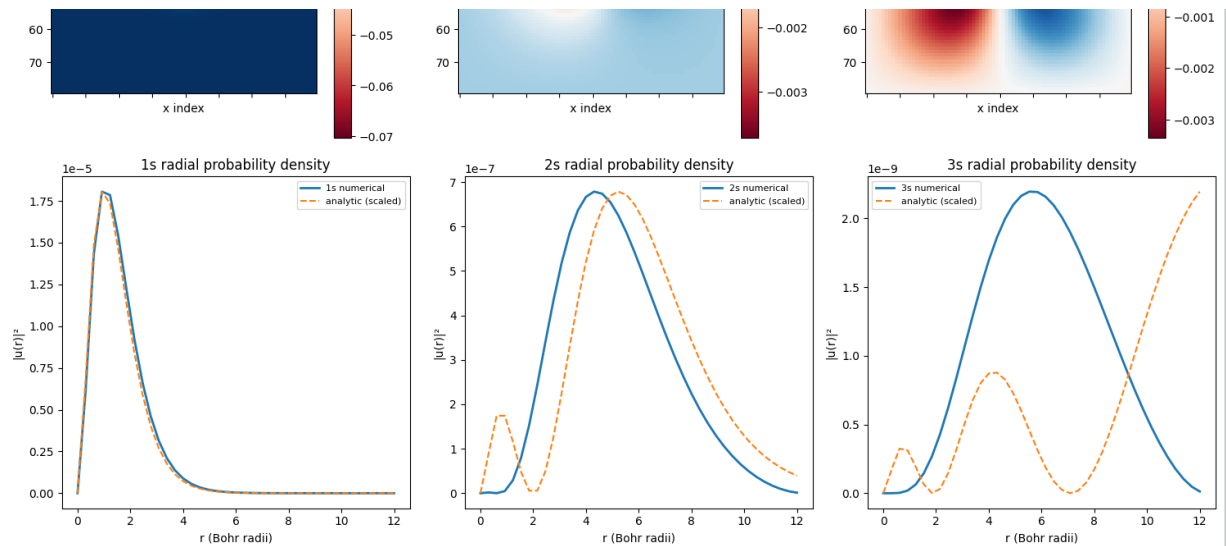
```

12 e_80, psi_80 = scipy.sparse.linalg.eigsh(H_3D, k=6, which='SA')
13 order = np.argsort(e_80)
14 e_80 = e_80[order]
15 psi_80 = psi_80[:, order]
16 print("Eigenvalues (eV):", e_80 * 27.21)
17
18 # 2-D slices: s-states at indices 0 (1s), 1 (2s), 5 (3s)
19 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
20 for pi, (si, lbl) in enumerate([(0, '1s'), (1, '2s'), (5, '3s')]):
21     A = psi_80[:, si].reshape(80, 80, 80)
22     im = axes[pi].matshow(A[40, :, :], cmap='RdBu')
23     axes[pi].set_title(f'{lbl} state (z=0 slice)')
24     axes[pi].set_xlabel('x index'); axes[pi].set_ylabel('y index')
25     plt.colorbar(im, ax=axes[pi])
26 plt.suptitle('2-D Slices of s-Type Eigenstates (g=80)')
27 plt.tight_layout()
28 plt.show()
29
30 # Radial probability density  $|u(r)|^2 = |r \cdot \psi(r)|^2$  along the axis
31 r_axis = np.linspace(0, 12, 40)
32 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
33 for pi, (si, n_val, lbl) in enumerate([(0, 1, '1s'), (1, 2, '2s'),
34     A = psi_80[:, si].reshape(80, 80, 80)
35     # Starting at index 40 (= origin) and going along z-axis
36     psi_line = r_axis * A[40, 40, 40:] / (4 * np.pi * np.sqrt(h
37     axes[pi].plot(r_axis, np.abs(psi_line)**2, lw=2, label=f'{l
38     # Analytic comparison (scaled to match)
39     r_safe = np.where(r_axis < 1e-3, 1e-3, r_axis)
40     u_ana = r_safe * radial_function(n_val, 0, r_safe)
41     peak_n = np.max(np.abs(psi_line)**2)
42     peak_a = np.max(u_ana**2)
43     if peak_a > 0:
44         axes[pi].plot(r_axis, u_ana**2 * peak_n / peak_a, '--',
45     axes[pi].set_xlabel('r (Bohr radii)')
46     axes[pi].set_ylabel('|u(r)|^2')
47     axes[pi].set_title(f'{lbl} radial probability density')
48     axes[pi].legend(fontsize=8)
49 plt.tight_layout()
50 plt.show()

```

Eigenvalues (eV): [-13.31463754 -3.3974114 -3.3974114 -3.3974114
-1.25351682]





✓ Answer (end)

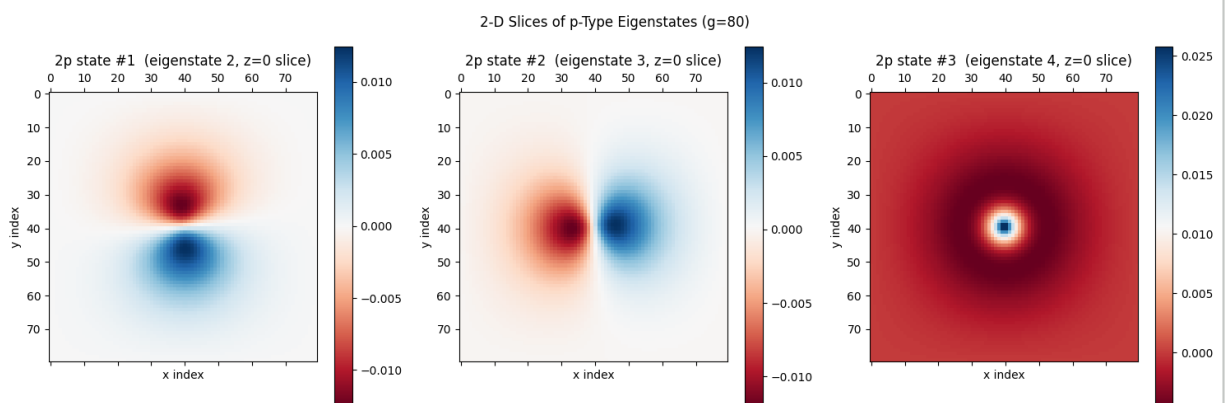
You can also look at the 2'nd eigenstate (and third and fourth) eigenstate on a 2d plot. This should show clearly the p-character of your state.

? Answer (start)

```

1 # p-states: indices 2, 3, 4 (three degenerate 2p states)
2 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
3 for pi, si in enumerate([2, 3, 4]):
4     A = psi_80[:, si].reshape(80, 80, 80)
5     im = axes[pi].matshow(A[40, :, :], cmap='RdBu')
6     axes[pi].set_title(f'2p state #{pi+1} (eigenstate {si}, z=0 slice)')
7     axes[pi].set_xlabel('x index'); axes[pi].set_ylabel('y index')
8     plt.colorbar(im, ax=axes[pi])
9 plt.suptitle('2-D Slices of p-Type Eigenstates (g=80)')
10 plt.tight_layout()
11 plt.show()
12
13

```



The three 2p states each display a clear nodal plane through the origin (dipole/figure-8 character) with each lobe aligned along a different Cartesian axis, in contrast to the spherically symmetric s-states.

✓ Answer (end)

✓ c. Three-dimensional plots

Now we'd like to make some three-dimensional plots.

Make some three-dimensional plots. To do this, you need to put your wave-function back on a grid - i.e. `psi[:,i].reshape(80,80,80)`.

Then use the following function

```
def PlotMe(psi):
    isomin=np.min(np.abs(psi)**2)
    isomax=np.max(np.abs(psi)**2)
    print(isomin,isomax)
    g=80
    p = np.linspace(-12, 12, g);          %# one dimension space lattice
    h = p[2] - p[1]                        %# lattice spacing
    L3=Laplacian(h,g)
    [X, Y, Z] = np.meshgrid(p, p, p);    %# three d
    # Create the isosurface
    isosurface = go.Isosurface(
        x=X.flatten(),
        y=Y.flatten(),
        z=Z.flatten(),
        value=(np.abs(psi)**2).flatten(), # Use the absolute value for
        #isomin=0.01**2, # Adjust the minimum intensity threshold
        #isomax=isomax, # Adjust the maximum intensity threshold
        surface_count=20, # Number of isosurfaces to display
        #colorscale='Viridis', # Choose a color scale
        opacity=0.5, # Set the opacity of the isosurface
        surface_fill=1.0,
        caps=dict(x_show=False, y_show=False)
    )
    fig = go.Figure(data=[isosurface])
    fig.show()
```

Compare these orbitals to the ones you got by directly plotting the analytic formula.

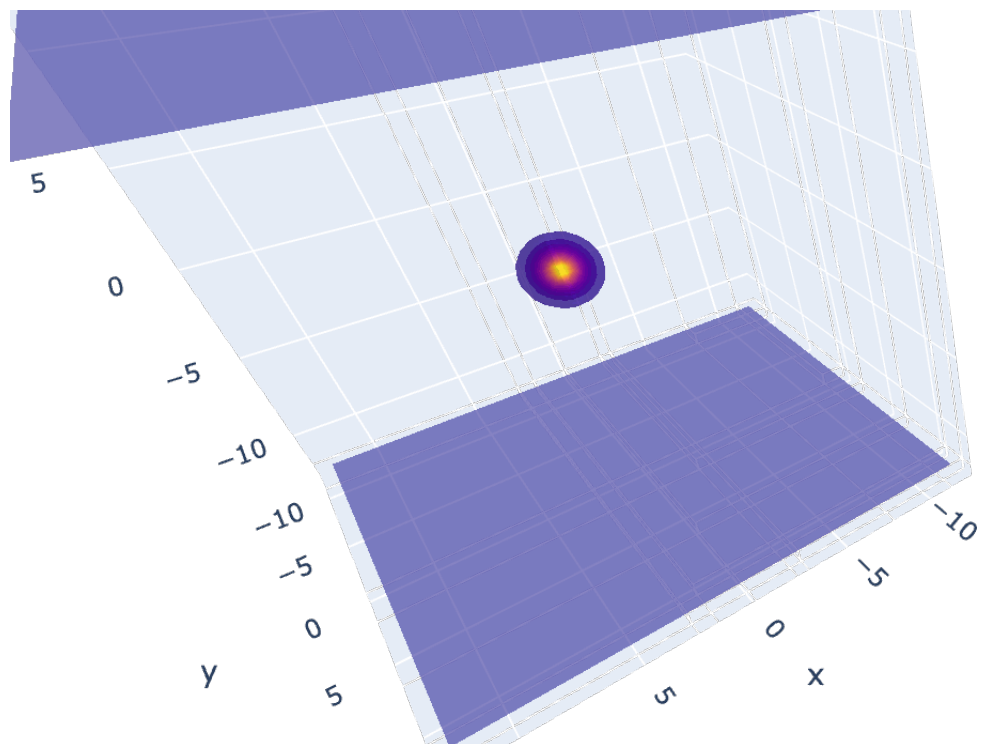
? Answer (start)

```

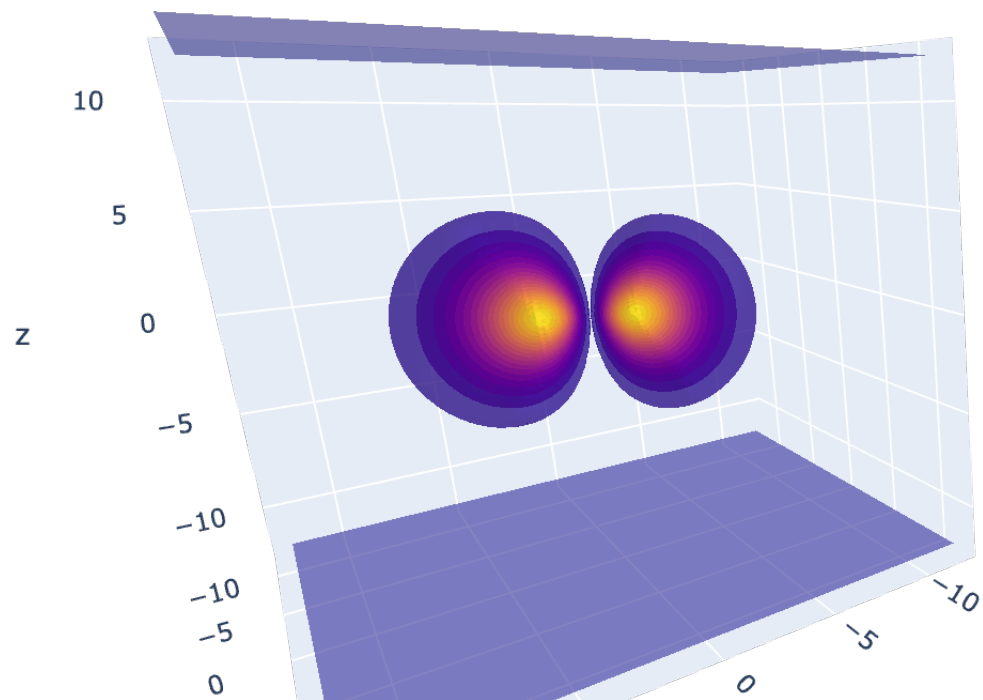
1 def PlotMe3D(psi_vec, g_pts=80, g_half=12):
2     """3-D isosurface plot of  $|\psi|^2$  using plotly."""
3     p = np.linspace(-g_half, g_half, g_pts)
4     X, Y, Z = np.meshgrid(p, p, p)
5     isosurface = go.Isosurface(
6         x=X.flatten(), y=Y.flatten(), z=Z.flatten(),
7         value=(np.abs(psi_vec)**2),
8         surface_count=20, opacity=0.5, surface_fill=1.0,
9         caps=dict(x_show=False, y_show=False)
10    )
11    fig = go.Figure(data=[isosurface])
12    fig.show()
13
14 # 1s state
15 print("3-D isosurface: 1s state")
16 PlotMe3D(psi_80[:, 0])
17
18 # One of the 2p states
19 print("3-D isosurface: 2p state")
20 PlotMe3D(psi_80[:, 2])
21

```

3-D isosurface: 1s state



3-D isosurface: 2p state



The 3-D isosurfaces confirm the spherical shell of the 1s orbital and the two lobes separated by a nodal plane of the 2p orbital, in excellent qualitative agreement with the analytically derived orbitals from Exercise 1.

✓ Answer (end)

Acknowledgements:

- Bryan Clark (original)

1 Start coding or generate with AI.

1 Start coding or generate with AI.